

一句话总结

这份 Introduction 讲义的核心是在回答一个问题：**大语言模型到底是什么，它是怎么从“预测下一个词”的概率模型，发展成今天可以问答、写作、翻译、推理、生成内容的通用智能系统的？**

它先从语言模型的数学定义讲起，再回顾语言模型的发展历史，最后解释为什么“大语言模型”值得单独开一门课：因为规模变大以后，模型能力、应用方式、社会影响和风险都发生了质变。

1. 什么是语言模型？

讲义一开始给出经典定义：

语言模型是一个对 token 序列分配概率的模型。

也就是说，给它一句话，比如：

the mouse ate the cheese

语言模型会判断这句话出现的概率有多大。概率越高，说明它越像自然语言；概率越低，说明它越不像正常语言。

通俗来说，语言模型本质上是在做一件事：

判断一句话“像不像人话”，以及在已有文字后面“最可能接什么”。

比如：

the mouse ate the cheese

比：

mouse the the cheese ate

更像正常英语，所以前者应该有更高概率。

这里的关键不只是语法。一个好的语言模型还需要隐含地掌握世界知识。比如：

the mouse ate the cheese
the cheese ate the mouse

这两句话语法都没问题，但前者更符合现实世界，所以模型应该给前者更高概率。

大白话解释：

语言模型不是简单背单词，而是在学习“语言规律 + 世界常识”。它要知道什么句子通顺，什么事情合理，什么表达更自然。

2. 自回归语言模型：一个词一个词往后生成

讲义重点介绍了 **autoregressive language model**，自回归语言模型。

它的基本思想是：

整句话的概率，可以拆成每一步“下一个 token 的概率”。

也就是：

第一个词出现的概率 ×

第二个词在第一个词之后出现的概率 ×

第三个词在前两个词之后出现的概率 ×

.....

所以生成文本时，模型不是一次性生成完整文章，而是：

1. 先看 prompt;
2. 预测下一个 token;
3. 把新 token 加入上下文;
4. 再预测下一个 token;
5. 不断重复。

例如：

the mouse ate

模型可能继续生成：

the cheese

大白话解释：

大模型写东西，就像一个非常强的“接龙高手”。你给它前半句，它根据上下文猜最可能的后半句。它不是脑子里提前有完整答案，而是一步一步往后续写。

3. Temperature：控制模型输出的随机性

讲义还讲了一个很重要的生成参数：**temperature，温度**。

它控制模型生成时有多“保守”或多“发散”。

当 $\text{temperature} = 0$ 时，模型每一步都选概率最高的词，所以输出最稳定、最确定。

当 $\text{temperature} = 1$ 时，模型按照原始概率正常采样。

当 temperature 越高时，模型越容易选择低概率词，输出就更有变化、更随机。

大白话解释：

temperature 就像创作时的“脑洞旋钮”。

- 温度低：像严谨的答题机器，稳，但可能死板。
- 温度高：像创意写作者，灵活，但可能胡说。
- 温度过高：可能开始乱飞。

所以写代码、做事实问答时通常希望温度低；写故事、广告文案时可以适当提高温度。

4. Prompt 和 Completion：提示词打开了通用任务能力

讲义强调，语言模型可以做 **conditional generation，条件生成**。

也就是：

给模型一个 prompt，它生成 completion。

例如：

Prompt: Frederic Chopin was born in

Completion: 1810, in Poland

这个形式非常重要，因为它意味着很多任务都可以被改写成“文本补全”。

比如：

问答可以变成：

Question: Where was Chopin born?

Answer:

类比推理可以变成：

sky: blue:: grass:

新闻写作可以变成：

Title: xxx

Article:

大白话解释：

Prompt 的伟大之处在于，它把很多复杂任务都统一成了一种形式：

我给你一段上下文，你接着往下写。

这就是为什么同一个大模型可以做翻译、问答、摘要、写代码、写邮件、改文章。它们在模型眼里，本质上都是“根据前文预测后文”。

5. 语言模型的历史：从信息论到 n-gram，再到神经网络

讲义回顾了语言模型的发展史。

最早可以追溯到 Claude Shannon 的信息论。他关注的问题是：

英语到底有多少信息量？能不能被压缩？

这里引出了 **entropy**，熵 和 **cross entropy**，交叉熵。

简单理解：

- 熵越低，说明语言越有规律，越容易压缩；
- 熵越高，说明语言越随机，越难预测。

后来出现了 **n-gram 模型**。

n-gram 模型的思想很简单：

预测下一个词时，只看前面固定几个词。

比如 trigram 只看前两个词：

$p(\text{cheese} \mid \text{the mouse ate the})$

会被近似成：

$p(\text{cheese} \mid \text{ate the})$

n-gram 的优点是计算便宜，可以在很大语料上训练。讲义提到，早期 5-gram 模型甚至可以训练在 2 万亿 token 上。

但它的缺点也很明显：

它只看很短的上下文，不能理解长距离依赖。

比如：

Stanford has a new course on large language models. It will be taught by ____

如果模型只看前几个词，就很难知道空格处应该和 Stanford 这个信息有关。

大白话解释：

n-gram 就像一个只看“最近几个字”的输入法。它很快，但记性太短。句子一长，它就忘了前面发生了什么。

6. 神经语言模型：更会泛化，但更贵

后来，Bengio 等人在 2003 年提出了神经语言模型。相比 n-gram，神经网络可以用向量表示词和上下文，因此更有泛化能力。

讲义总结了一个关键对比：

模型	优点	缺点
n-gram 模型	计算便宜、可扩展	统计效率低，长上下文能力弱
神经语言模型	泛化能力强、统计效率高	训练计算成本高

之后又出现了 RNN、LSTM 和 Transformer。

RNN/LSTM 理论上可以依赖完整上下文，但训练困难。

Transformer 虽然仍然有固定上下文窗口，但更容易并行训练，更适合 GPU，因此成为后来大语言模型的主流架构。

大白话解释：

n-gram 是“查表式记忆”，神经网络是“学规律”。

查表便宜但死板，神经网络贵但聪明。

随着 GPU 和深度学习发展，后者终于变得可行。

7. 为什么需要专门研究“大”语言模型？

讲义接着问：语言模型已经存在很久了，为什么还要专门开一门大语言模型课？

答案是：**规模改变了模型的性质。**

2018 年之后，模型参数规模迅速增长：

- ELMo: 9400 万参数
- GPT: 1.1 亿参数
- BERT: 3.4 亿参数
- GPT-2: 15 亿参数
- GPT-3: 1750 亿参数
- Megatron-Turing NLG: 5300 亿参数

讲义强调，模型变大之后，不只是性能更好，而是出现了 **emergent behavior**，**涌现行为**。

所谓涌现，就是：

原来模型做不到的能力，在规模变大后突然出现。

例如，GPT-3 可以通过 prompt 做问答、类比、文章生成，甚至出现 in-context learning。

大白话解释：

小模型像一个增强版输入法；

大模型开始像一个通用任务处理器。

它不只是“更会接话”，而是开始能“理解任务格式并完成任务”。

8. In-context Learning：不用重新训练，只靠例子学任务

讲义特别强调了 GPT-3 的一个重要能力：**in-context learning**，上下文学习。

传统监督学习是：

1. 给模型一批训练样本；
2. 用梯度下降训练模型；
3. 得到一个专门模型。

但 in-context learning 是：

1. 不改模型参数；
2. 只在 prompt 里给几个例子；
3. 模型就能模仿这些例子的格式完成新任务。

例如：

Input: Where is MIT?

Output: Cambridge

Input: Where is University of Washington?

Output: Seattle

Input: Where is Stanford University?

Output:

模型可能输出：

Stanford

大白话解释：

这就像你不需要重新训练一个员工，只要给他看几个样例，他就能明白你想要什么格式。

这也是大语言模型区别于传统 NLP 模型的关键能力之一：

它不只是被训练出来完成一个固定任务，而是可以在上下文里临时“读懂任务”。

9. 大语言模型的现实应用

讲义指出，大语言模型已经深刻影响 NLP 研究和工业应用。

在研究中，情感分类、问答、摘要、机器翻译等大量任务的 SOTA 系统都基于语言模型。

在工业中，语言模型也被用于：

- 搜索；
- 内容审核；
- 写作辅助；
- API 服务；
- 各种语言相关创业产品。

但讲义也提醒，真实工业系统往往不是一个裸模型直接输出，而是经过：

- 微调；
- 蒸馏；
- 多模型组合；
- 场景化优化；
- 工程系统包装。

大白话解释：

真实产品里的大模型不是“一个 GPT 直接裸奔”。

它通常被包在一整套系统里：前面有检索、规则、权限控制，后面有过滤、校验、格式化、风控。

10. 大语言模型的风险

讲义最后重点讲了大语言模型的风险，这部分非常重要。

1. 可靠性问题

模型可能给出错误答案，而且错误答案看起来很像真的。

比如问：

Who invented the Internet?

模型可能错误回答：

Al Gore

这在医疗、法律、金融等高风险领域尤其危险。

大白话解释：

大模型最大的问题不是“不知道”，而是“不知道还说得很像知道”。

2. 社会偏见

模型从互联网数据中学习，而互联网本身有偏见，所以模型也可能继承偏见。

比如职业和性别的关联：

The software developer finished the program. He celebrated.

The software developer finished the program. She celebrated.

模型可能对两句话给出不同概率，反映训练数据中的性别刻板印象。

大白话解释：

模型不是价值中立的。它学的是人类文本，而人类文本里有偏见、刻板印象和不平等。

3. 毒性内容

互联网语料里有大量攻击性、歧视性、仇恨性内容。模型可能在某些 prompt 下生成 toxic output。

这对聊天机器人、写作助手等产品都有风险。

4. 虚假信息

讲义提到，GPT-3 可以很容易编造一篇看起来真实的新闻文章。

这意味着大语言模型可能被用于：

- 虚假新闻；
- 舆论操纵；
- 自动化水军；
- 政治宣传；
- 诈骗内容生成。

大白话解释：

以前制造高质量假内容需要人力和语言能力。

现在模型可以低成本批量生成流畅、有说服力的假内容。

5. 安全风险

因为模型训练数据来自公共互联网，攻击者可能故意发布某些网页，让它们进入训练集，形成 **data poisoning，数据投毒**。

比如让模型看到某个品牌名时，总是生成负面情绪文本。

6. 法律和版权问题

模型可能训练在版权数据上，也可能在生成时复现版权文本。

讲义举了 Harry Potter 的例子：如果 prompt 给出开头，模型可能继续生成原文内容。

这引出问题：

- 训练使用版权数据是否属于 fair use?
 - 用户生成了版权文本，责任算谁的?
 - 模型服务商是否要承担责任?
-

7. 成本、环境和开放性问题

大模型训练非常昂贵。讲义提到 GPT-3 的训练成本估计约 500 万美元。

成本高带来两个后果：

第一，能训练大模型的机构越来越少，技术集中在少数大公司手里。

第二，训练和推理都需要大量 GPU 和能源，带来环境影响。

大白话解释：

大模型不是普通实验室随便能玩的东西。它越来越像“重工业”：需要资金、算力、工程团队和数据资源。

11. 这门课的结构：像洋葱一样一层层剥开

讲义最后介绍了 CS324 的课程结构，称之为像洋葱一样逐层深入。

第一层：研究大模型行为

先把模型当黑箱，只通过 API 观察它能做什么、不能做什么、会出什么问题。

这类似于：

| 像生物学家研究一个生物一样，先观察行为。

第二层：研究训练数据

再往里看模型背后的数据，包括：

- 数据来源；
- 安全；
- 隐私；
- 版权；
- 偏见。

第三层：研究模型构建

继续深入模型核心，包括：

- 架构；
- 训练算法；
- 优化方法；
- 规模化训练。

第四层：超越语言模型

最后把语言模型放到更大的 foundation models 框架下看。

因为 token 不一定只是自然语言，也可以是：

- 代码；
- 图像；
- 音频；
- 其他离散符号。

大白话解释：

这门课不是只教你怎么调 API，而是从外到内理解大模型：

先看它表现 → 再看它吃了什么数据 → 再看它身体结构 → 最后看它在整个 AI 技术体系中的位置。

最重要的 8 个概念

如果你要把这份 Introduction 真正学懂，我建议重点抓住下面 8 个概念：

概念	你需要理解什么
Language Model	本质是对 token 序列建模的概率分布
Autoregressive Model	一个 token 一个 token 地预测下一个 token
Prompt / Completion	大多数任务都可以转成条件生成
Temperature	控制输出随机性和创造性
Entropy / Cross Entropy	用信息论衡量语言可预测性
N-gram	早期语言模型，只看短上下文
Transformer / Neural LM	用神经网络学习语言规律，成为现代大模型基础

概念	你需要理解什么
In-context Learning	不改参数，只靠 prompt 示例临时学任务

总结

大语言模型不是突然出现的魔法，而是语言模型长期发展的结果。它的底层目标仍然是预测 token，但当模型规模、数据规模和计算规模足够大以后，预测 token 这件事产生了通用任务能力。

大语言模型的核心并不是“会聊天”，而是一个基于条件概率的序列建模系统。它通过自回归方式预测下一个 token，而 prompt engineering、问答、摘要、翻译、代码生成，本质上都是 conditional generation 的不同形式。真正的变化来自 scale：当模型参数、数据和计算量扩大后，模型出现了 in-context learning 等涌现能力，使语言模型从传统 NLP 系统中的一个组件，逐渐变成可以独立完成多类任务的通用基础模型。

这句话基本就把 Introduction 的核心逻辑讲清楚了。